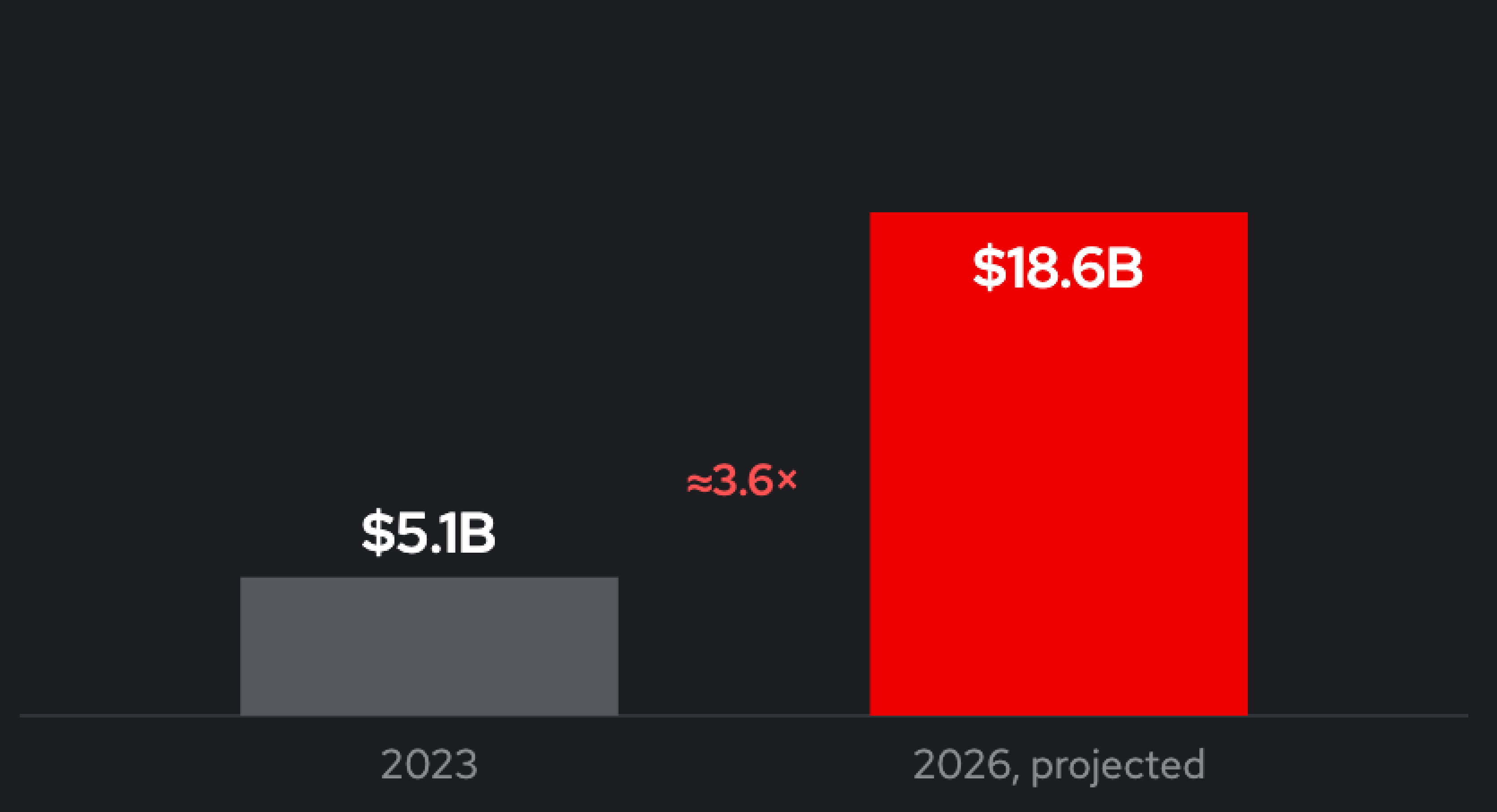
THE QUESTION ON THE TABLE
Is there real demand for this? And is it a content-creation studio, or another AI playground?

A studio market, already large, repricing quarterly

7 MONTHS	12 MONTHS	3 MONTHS
$1.3B → $5.4B	**$20M → $700M**	**42×**
Higgsfield valuation	annualized revenue	agentic usage

- Category **~$5B in 2023 → ~$18.6B** projected for 2026.
- **Two companies passed $5B** valuation inside one year: Higgsfield and Runway.
- **Enterprise is the buyer**: business went from under a quarter to the majority of Higgsfield revenue in 7 months.
- **390 of the Fortune 500** use Higgsfield for visual production.
- **Agents are the next wave**: 42× agentic-usage growth in 3 months.

PROFESSIONAL AI IMAGE & VIDEO, CATEGORY SIZE



Demand is real, it is enterprise, and it is repricing quarterly.

Reuters · FT · PR Newswire, Aug 2026 · Runway Series E, Feb 2026 · AI-video market trackers, 2026 · docs/14-market.md

2 · WHAT THIS IS, IN YOUR TERMS

THE QUESTION ON THE TABLE

What is graph-based inference, and how is it different from serving a text model?

A graph is a spreadsheet. A model is a file the job loads.

IF YOU KNOW VLLM

In vLLM the model is a **deployment**: resident weights, one endpoint. In ComfyUI the model (checkpoint) is **data**: a file this job loads and the next job replaces.

IF YOU DON'T

A prompt box is a **calculator**. A node graph (workflow) is a **spreadsheet**: change one cell and only the dependents recompute.

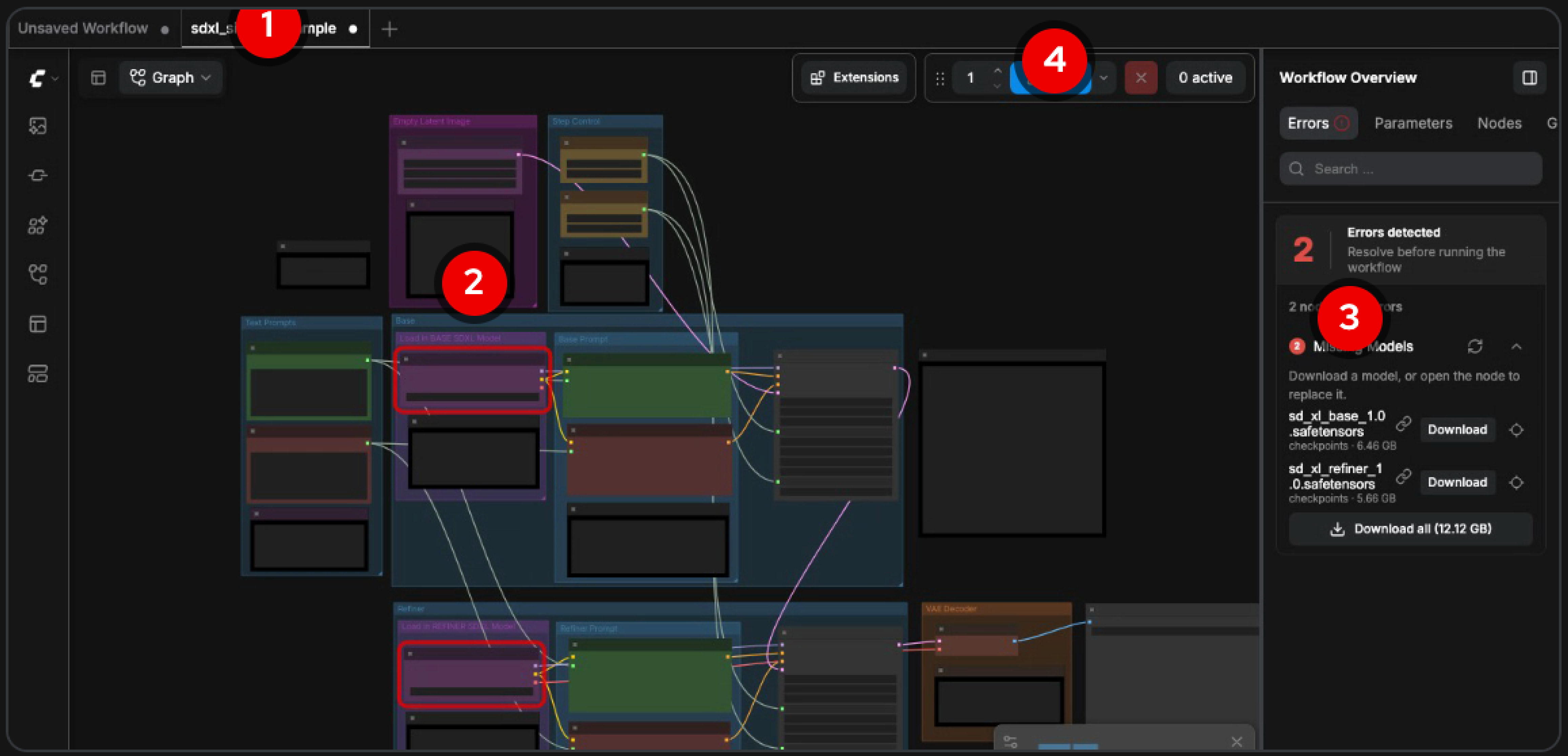
- **One job**: 10+ reference images, text, a few LoRAs, two or three base models → a 40-node graph → one image.
- **Most nodes are not inference.** They mask, composite, and wire control signals.
- **Templates** are graphs someone else built, or you build yourself. The team's catalog of models only grows.

TEXT SERVING (vLLM)

prompt — one resident model — tokens

ONE GRAPH JOB (ComfyUI)

10+ images | text | LoRAs | 2–3 models — **40-node graph, per job** — one image



1 template 2 a model (checkpoint) loader node 3 missing-models panel

4 Run: the only GPU step

A graph is a spreadsheet; a model is a file. Everything after this follows.

Vocabulary used from here on: model (checkpoint) · graph (workflow) · canvas (frontend) · worker (pod) · docs/13-vllm-omni.md

THE QUESTION ON THE TABLE

Why ComfyUI, against the commercial tools?

The category's open-source leader, missing only its enterprise story

- **4M users, 150k downloads a day, $500M valuation**, 60,000+ community nodes.
- In production at **Amazon Studios, Apple, Autodesk, Netflix, Nike, Tencent, Ubisoft**.
- Undercutting a price umbrella with open source on a supported platform is **the house strategy**.
- **“ComfyUI artist” is a job title.** Behind the first mostly-AI Super Bowl ad.

Amazon Studios · Apple · Autodesk · Netflix · Nike · Tencent · Ubisoft · Pixomondo · HP · Lucid

	CREDIT-METERED SAAS Higgsfield · Krea · Runway · Midjourney	COMFYUI, ON THIS POOL
Price	$79–215 per seat per month; video by the second	Open source; ten designers ~$120–950/mo of GPU
Models	The vendor's catalog	Any model (checkpoint), yours or public; 40–120 GB team libraries
Nodes	The vendor's pipeline	60,000+ community nodes, arbitrary Python
Your data	Prompts, models, outputs in the vendor's cloud	**Your VPC, your SSO**; NDA studios can use it
Who buys	Studios, agencies, brand teams	The same buyers, on hardware they already have

The tool professionals already use, with no enterprise deployment story. Until now.

THE QUESTION ON THE TABLE

Can the expensive machine under each creator's desk be broken up?

A card per person, idle most of the day, or one unauthenticated box

PER SEAT

$6–8k

a capable desktop

TEN PEOPLE, CLOUD

~$7,100/mo

a card each, running

- **A $6–8k desktop per seat**, idle most of the day. Cloud equivalent: a card each.
- **Four of five loop steps never touch a GPU.** Only the run step does.
- **One shared box**: a whole team on one unauthenticated process; any custom node runs as the server.
- **NDA studios** cannot send frames to a vendor cloud at any price.

THE DESIGNER'S LOOP, EVERY DAY

Load a template (no GPU) → **See missing models** (no GPU) → **Load the models** (no GPU) → **Wire & fix the graph** (no GPU) → **Run** (~2 min on a GPU)

Setup between renders takes minutes; the render takes about two. A designer needs a card a small fraction of the day.

Four of five steps never touch a GPU.

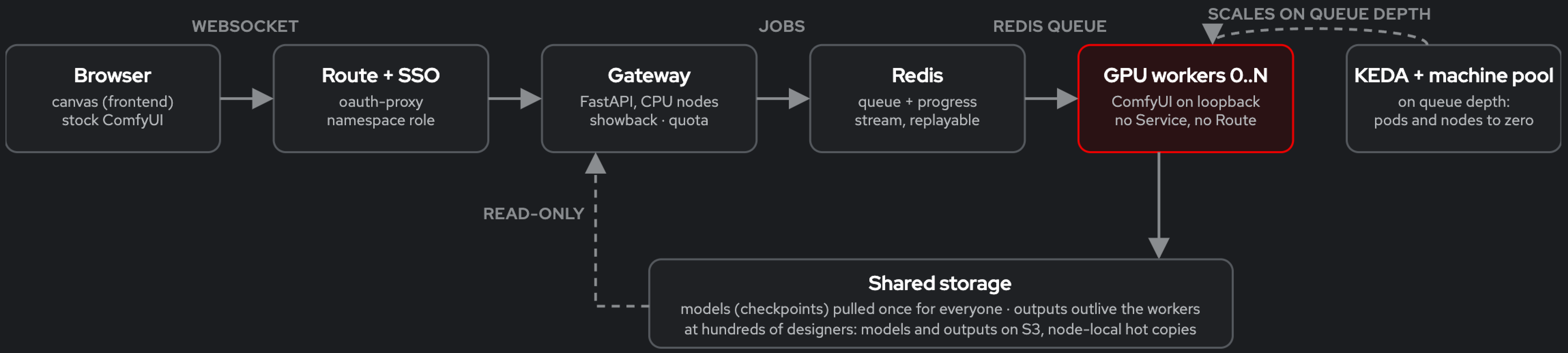
THE QUESTION ON THE TABLE

GPUs to the datacenter, frontend untouched?

Same application, unchanged. Only the silicon moved.

- **The application is untouched**: stock ComfyUI at a pinned commit, not a wrapper. Same canvas, same loop: template, missing models, one click, run.
- **Models pulled once** to shared storage at datacenter bandwidth. They outlive the workers.
- **Only GPU access changes**: efficient, scalable, and larger than the card under the desk.
- **Workers scale from zero to N** on queue depth, pods and nodes both.

THE SHAPE: HUB AND SPOKE, WORKERS UNREACHABLE BY DESIGN



Same application. Only the silicon moved.

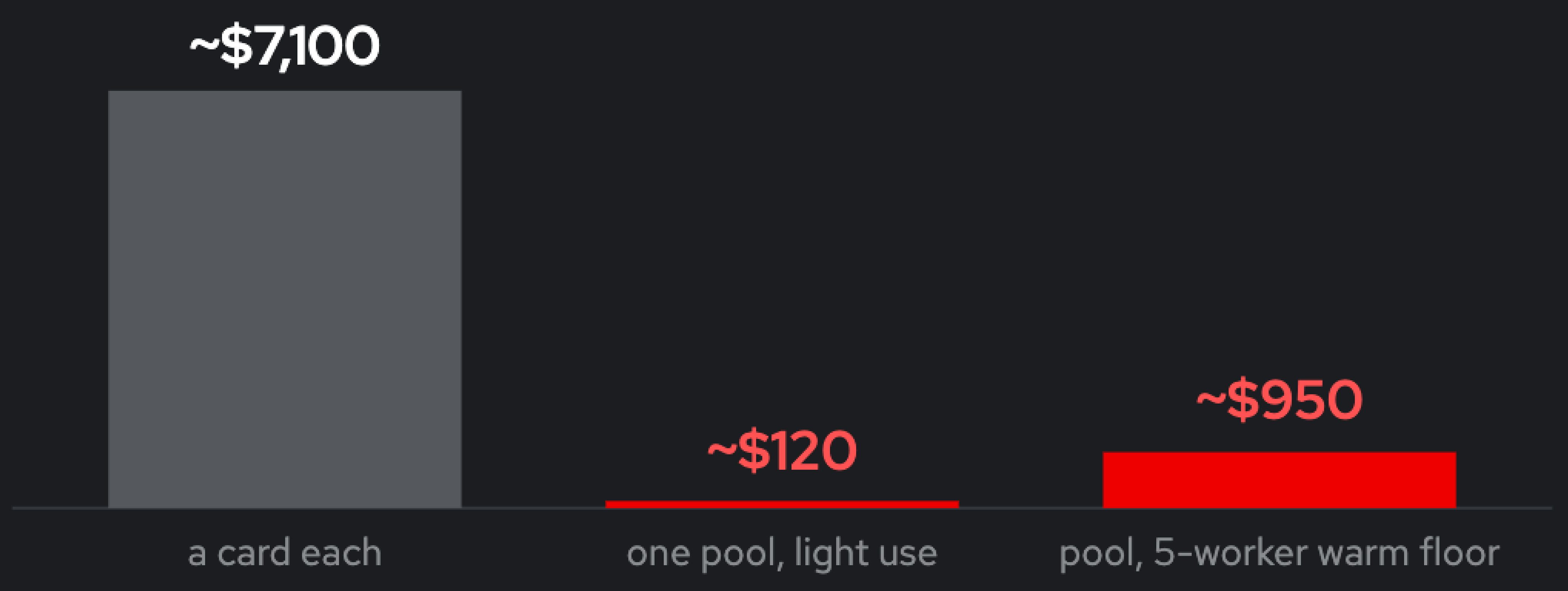
THE QUESTION ON THE TABLE

What actually drives GPU cost for this workload?

Pooling is queueing, not division

- **Ten designers**: ~$7,100 a month as a card each; **$120–950** as one pool.
- **Savings widen with headcount**: 33% at five people, 64% at thirty.
- **Scale-to-zero means zero nodes**, not zero pods. Only the node layer saves money.

TEN DESIGNERS, MONTHLY GPU LINE



CONCURRENT WORKERS GROW FAR SLOWER THAN HEADCOUNT

TEAM	CONCURRENT WORKERS	POOL, WARM 9–6	A CARD EACH	SAVINGS
5	3	$3.99/hr	$5.94/hr	**33%**
10	5	$5.94/hr	$10.82/hr	**45%**
15	6	$6.92/hr	$15.70/hr	**56%**
20	8	$8.87/hr	$20.58/hr	**57%**
30	10	$10.82/hr	$30.34/hr	**64%**

Both columns include the $1.06/hr cluster floor and $0.976/hr per L4. Average queue wait held under ~30 s.

WHERE THIS LOSES, HONESTLY

Cold start of 8–17 min when nothing is warm (the warm floor is a setting, ~$190/mo per card). **Custom nodes** need an image rebuild, not a click. Both priced in the README, “Where this loses”.

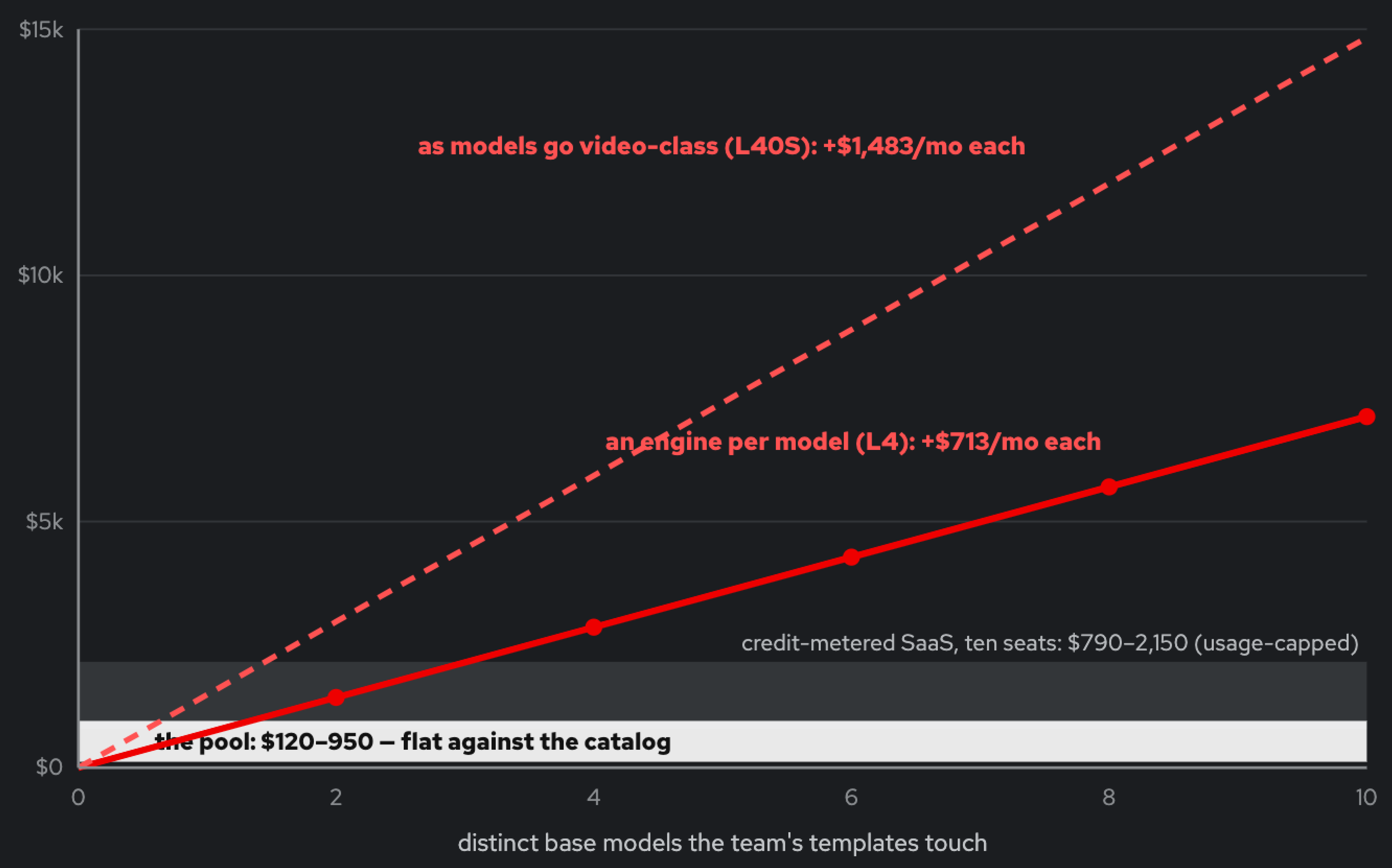
Doubling the team does not double the overlap.

THE QUESTION ON THE TABLE

Why not simply serve every model behind an engine?

Residency scales with catalog size. The pool scales with concurrency.

TEN PEOPLE, GROWING MODEL LIBRARY — MONTHLY GPU COST



- **Residency** scales with catalog size. **The pool** scales with concurrency.
- The marginal model costs **a machine** on a serving tier and **disk** on the pool.

The marginal model costs a machine on an engine, a disk on the pool.

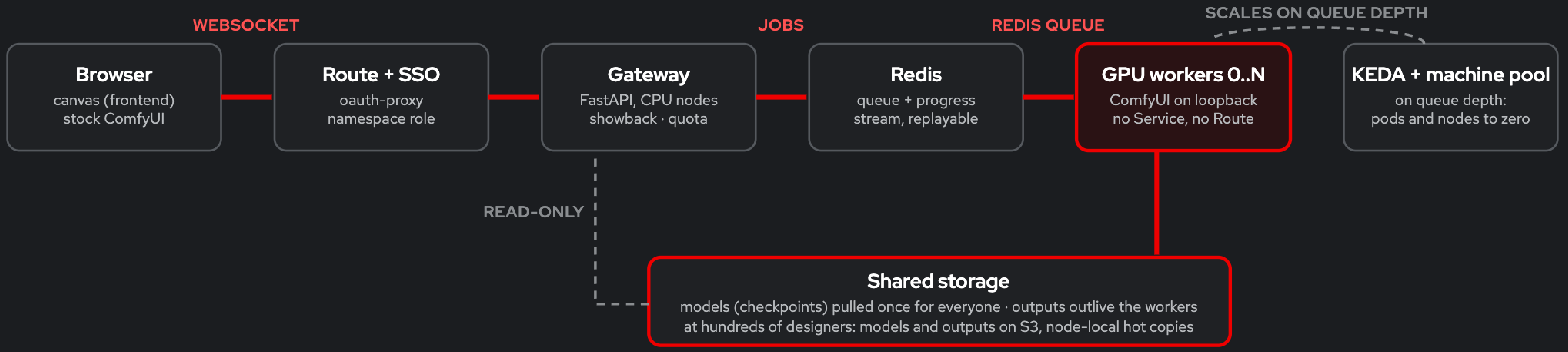
THE QUESTION ON THE TABLE

What does this provide beyond the existing Omni–ComfyUI integration?

The production layer: queue, isolation, storage, scale

- **Browser → gateway over WebSocket, gateway → workers over a Redis queue** that places each job by declared VRAM tier (L4 today; L40S, H100 as jobs grow). No browser ever touches a worker, so workers can vanish mid-job and scale to zero.
- **Storage not shared**: models to S3 with node-local hot copies, outputs to S3. Same shape, several hundred designers, low hundreds of GPUs, no rewrite.
- **Common storage, least friction**: a model pulled once is there for everyone. Outputs outlive the worker and the cluster.
- **Workers unreachable by design**: ComfyUI on loopback, no Service, no Route, default-deny. Cluster SSO, audit log, showback, quota.

THE SAME SHAPE — THE TRANSPORT HIGHLIGHTED



Workers are unreachable, replaceable, and scale to zero.

github.com/redhat-et/comfyui-on-openshift · docs/06 architecture · docs/11-scaling.md "The bottleneck is storage, not GPUs"

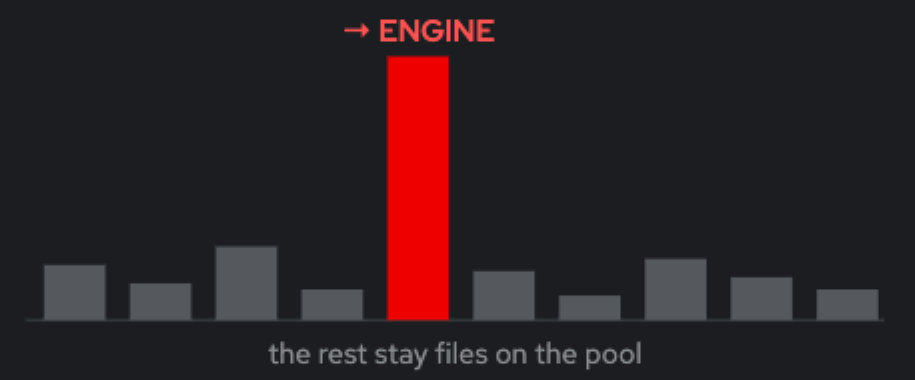
THE QUESTION ON THE TABLE

Where does the strategic inference engine fit? Does any of this stray into the application layer?

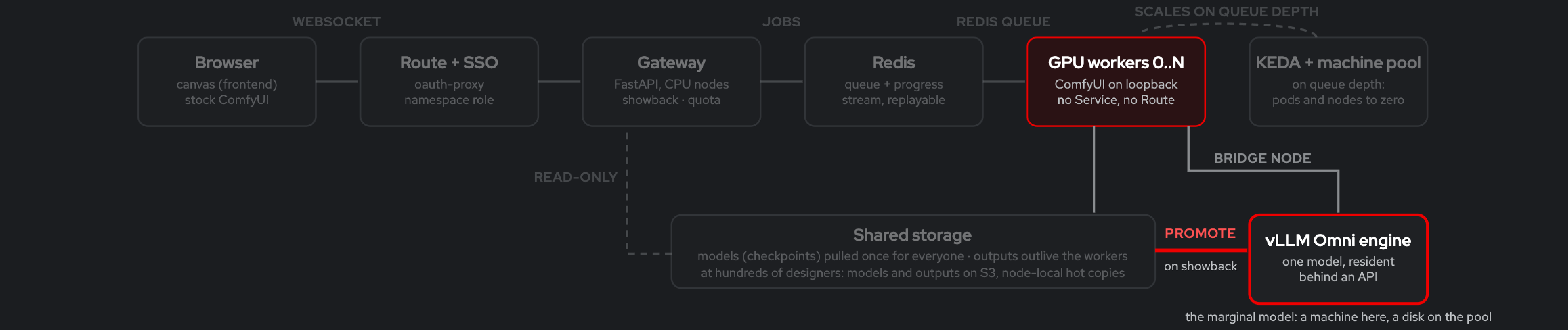
Two tiers, one stack. Showback decides when a model moves.

- **The engine is the throughput tier. The pool is the flexibility tier.**
- **Promote** a model when showback shows it dominating GPU-seconds at sustained duty.
- **ComfyUI stays upstream's app.** This repo is deployment wiring only.
- **Omni joins the canvas as one more node.** The bridge already does this, so the two tiers compose today with no new integration.

SHOWBACK: GPU-SECONDS PER MODEL, ONE MONTH



THE SAME SHAPE — THE ENGINE BESIDE THE POOL



Two tiers, one stack. Showback decides when a model moves.

THE QUESTION ON THE TABLE

Why is this ours to do?

Red Hat is the best place to run ComfyUI

- An **$18B category** whose open-source leader has no enterprise deployment story. Untapped, and tripling in three years.
- **OpenShift already had the parts**: NVIDIA GPU Operator, cluster SSO, machine pools that scale to zero, network policy, audit. This repo is the wiring.
- **The same application, unchanged**, fitted to real studio workflows, cost-effective on constrained GPUs.
- **Inference at Red Hat needs a graph-input answer** beside the text-input one. The pool is that answer, and the on-ramp to Omni.

ComfyUI
upstream, pinned, unchanged

The pool — this repo
queue · isolation · storage · showback

OpenShift
SSO · machine pools to zero · network policy · audit

NVIDIA GPU Operator
L4 today · L40S, H100 as jobs grow

The graph-input half of inference, on the platform we already sell.

Every way to give a team ComfyUI, side by side

	COST AT 10 USERS	SSO + AUDIT	RCE CONTAINMENT	CUSTOM NODES	WHO OPERATES IT	YOUR VPC
Shared box, `--multi-user`	one card, always on; ten queue on one process	none	none: any node runs as the server	live, for everyone	whoever owns the box	yours
A pod or VM per person	a card each, ~$7,100/mo	whatever you bolt on, ten times	one blast radius per person	per person, live	you, ten times	yours
Serverless ComfyUI (RunPod, Modal)	per-second with markup	the vendor's keys, not your IdP	the vendor's	your image; rebuild	the vendor	no
Hosted ComfyDeploy / ViewComfy / Comfy Cloud	per seat or per second	the vendor's login	the vendor's	the vendor's catalog	the vendor	no
Credit-metered SaaS	$79–215/seat/mo; video by the second	vendor login; SSO on enterprise plans	n/a, closed	no	the vendor	no
Generic Kubernetes Helm chart	a card per replica, always on	whatever ingress auth you add	a reachable Service by default	your image	you: cluster, driver, app	yours
KServe / Ray Serve	a resident GPU per served model	the platform's	the serving container	the model is the unit, not the graph	you, plus the serving stack	yours
This repo	**~$120–950/mo** + $1.06/h floor	**cluster SSO; every grant audited**	**loopback, no Service, no Route, default-deny**	baked into the image; a rebuild	**one namespace**; the platform pages for the rest	**yours**

Every rung is one command with a known return time

SMALLEST USEFUL CLUSTER: ONE COMFYUI WORKER ON AN L4, US-EAST-2

STATE	$/HOUR	GETTING BACK
Running	~2.04	–
GPU parked (`make park`)	~1.06	~5 min
Cluster torn down (`make down`)	~0.05	~15 min

RATES BEHIND EVERY NUMBER IN THE DECK

g6.xlarge (L4 24 GB) $0.976/h all-in: $0.805 to AWS, $0.171 to Red Hat. **g6e.xlarge** (L40S) $2.03/h. Cluster floor $1.06/h ≈ $775/mo. Per L4 per month: **$713**.

HABIT, MONTHLY

PATTERN	MONTHLY
Left running 24/7	~$1,490
Weekdays 9–6, `make park` nightly	~$965
Weekdays 9–6, `make down` nightly	**~$425**
Up one day a week	~$114

A single cron line is a **~70% cut**. The 15-minute rebuild is what makes teardown a habit rather than a heroic act.

THE WARM FLOOR

WARM_WORKERS=5, weekdays 9–6: ~$190/mo per warm card. KEDA takes the max across the cron and queue triggers, so a busy afternoon still bursts past the floor.

Every number on these slides is backed in the repository

- **Higgsfield** valuation, revenue, agentic growth, customer mix, Fortune 500 count: Reuters, Financial Times, PR Newswire, Aug 2026.
- **Runway** Series E, $315M at $5.3B, ~$300M annualized revenue: press coverage, Feb 2026.
- **Category size** $5.1B (2023) → $18.6B (2026, projected): AI-video market trackers, 2026.
- **ComfyUI** raise and valuation: Craft Ventures round, Apr 2026; users, downloads, node count: raise announcement; adopters: comfy.org.
- **Comfy MCP**, official hosted + open-source local server: comfy.org/mcp, June 30, 2026.
- **SaaS pricing**: vendor team-tier pricing pages, 2026; Runway credit schedule.
- **Cost model**, every rate and monthly figure: `docs/02-cost.md`; sizing tables: README “Sizing the pool”.
- **Catalog curve** and the composition rule: `docs/13-vllm-omni.md`.
- **Architecture**, WebSocket and Redis streams, unreachable workers: `docs/06-enterprise-architecture.md`.
- **Scaling to hundreds**, storage ladder: `docs/11-scaling.md`.
- **Market case**: `docs/14-market.md`. **Comparison table**: README “Against the alternatives”.
- **The long deck**, six slides with the demo on video: `docs/pitch/`.